

PROJECT PREP

Title : Code mixed Humour Detection (Romanised Telugu-English)

Goal :

1. Built a code mixed (Telugu-English) Dataset
2. Perform NLP's on the dataset
3. Obtain scores & observe how well the model works.

Why : Romanised Telugu is most widely used language in social media platforms like Twitter, Instagram, facebook & others.

And there is no research or datasets that contain code mixed data in Telugu-English. So our dataset will be the first dataset in these 2 languages.

Why
Humor :

~~And~~ Humor (or) Jokes is deeply a Contextual & cultural

Sometimes, Hatred comments (or) Negative comments are often sent through masked humor. So there is a need to detect pure humor.

Why not Sentiment Analysis?

A: Humor often contradicts Sentiment.

Humor also requires understanding cultural context, word play & Sarcasm; which Sentimental models can't capture.

What we achieved?

- No labelled dataset exists for humor detection in this pair, So we created it.
- Built a Humor Detection System for low language resource.
- Implemented a Hybrid Model (Transform + Rule Based System)
- Improved Accuracy (92.41%) than other Telugu understandable Transformers.

Generate Dataset

Technical & Non-technical.

Describe your dataset?

A: Our Dataset size is 10,000 texts combined of Humours & Non humours Sentences.

It is a Balanced dataset (5K humours + 5K non-humours)

What is your Source Dataset? *

A: There is a kaggle dataset that we took reference from and generated a Synthetic dataset.

Why only specific categories like College, family & others..?

A: Because these are the most daily conversational topics in telugu households & discussions. & These make us dataset feel natural.

Your Data feels Synthetic...? Explain?

Yes, it feels synthetic because it is generated out of code, because we have less time & it is ~~produced from a~~

template driven and Synthetic data is widely used in

NLP, when low resource languages lacks annotated data. They also help in initial experimenting... & Controlled evaluation

Some facts:-

1. Template Based generation + randomness + linguistic variation.

We used → 1 Word Lists: provide vocabulary

→ male names, female names, places, colleges, subjects, foods, emotions & more.

→ Spelling variants: needed because romanised telugu has no standard spellings;

→ Helper Functions: $VL()$ - spelling variations
 $p()$ - picks random words.

→ Emotions Injectors: laugh(), cry(), angry()...
makes text look like natural & detect emotions.

Code make Sures:- 1. Balanced dataset

2. Avoid duplicates

3. Randomly shuffles

Humor Category

→ college
family
Sarcasm
Tollywood
Relationship
Daily life.

* Each has
random filling
→ contains 10-12
templates.

Non-Humor Category.

Daily statements.
Instructions
plain options
News.
Facts.

Used Random module to avoid repetition.

Shuffling data ensures randomness during training.

Template Based Generation approach combined with
Randomness & Spelling variation modelling to Simulate
real world language.

Core innovation:- linguistic noise modeling.

→ Random module introduce dataset diversity.

→ world lists, Spelling variants simulates real world
noisy text.

Why probability:- Some Sentences has emoji & Some don't

Probability ensures Randomness
Here in the dataset,

60% Sentences has Emojis

40% don't.

If ~~the~~ probability is less than 0.6

$> 0.6 \rightarrow$ Emoji is added.

$\rightarrow$ Randomness gives probability values b/w 0 & 1.

$\hookrightarrow$ is introduced to control randomness,

Without probability, dataset becomes deterministic & leads to overfitting.

`random.seed(99)`: Samples can be re-generated

consistently for de-bugging, Evaluation

without fixed seed, each run produces different dataset, making dataset inconsistent,

* purely generated Synthetic dataset created from python code.*

Dataset details: - Cross-validation V_2 + V_3 .

Training: 80%.

Validation: 10%.

Testing: 10%.

V_2 dataset performs better

(mainly F1 score)

→ improved model learning + generalization

V_3 has Superior performance

→ V_3 Captures richer & more diverse

humane patterns.

So, we choosed V_3 for better performance.

MAIN CODE:

app.py.

Imports : gradio, deep learning + softmax,
(UI) (torch)

Transformers (MuRIL), AutoTokenizer

Auto Model for Sequence

classification.

Regex → pattern matching slang, sarcasm.

difflib → fuzzy matching.

Heuristic Engine → core logic.

Humor ~~at~~ Signals → 1. Emojis — Adds humor score.

2. Serious Emojis — reduce humor score.

3. English slang — Ex: lol, bruh.

no cap, touchwood.

4. Telugu slang — single, multiple.

5. Hyderabadi slang — slanged Hindi

6. Sarcasm patterns — detects indirect humor.

→ Uses MuRIL transformer model (deep learning model.)

→ Rule Based "heuristic Engine"

↓
Core logic

MuRIL + heuristic Engine

Semantic meaning + Stylish & cultural cues.
(MuRIL) (heuristics).

Operations :

1. Cleaning the data (removing noise)
2. Lowercasing. (capital cases may be captured as Non-humors.)
3. Normalization (lower case, removing noise, standardizing)
4. Tokenization (splits the text into smaller units).

MuRIL: Multilingual Representations for Indian Languages.

→ developed by Google.

→ Based on BERT architecture.

→ Consists of :
✓ Multi-Head Self attention (long range dependency)
✓ Feed Forward Neural Network.

✓ Helps in: Context, word relationships, Sentence meaning, Phonetic Similarity.

Why MuRIL :- • MuRIL is exactly trained for all 17+ Indian languages.

Transformer

Based NLP model

- It can understand both native & Romanised Text

- ✓ Handles variations in Spellings.

- ✓ Better than generic models.

(Because generic models mostly combined with Asian languages (or) they have less Romanised script).

- It is purely Indian specific

- ✓ → Includes Transliteration.

- ✓ → understands Telugu language well.

- ✓ It can also handle Code mixing + Roman Telugu +

Slang variation.

What is a Transformer?

A. A transformer is a deep learning model architecture used in NLP that process input data using a mechanism called Self-attention, allowing it to understand all parts of input simultaneously.



MuRIL exactly can't catch Emojis, slangs, & Repeated letters, Humour patterns. It struggles with Non-linguistic Cues.

That is why we used: **Heuristic Engine**.

In code: `Compute_humor_score()` $\Rightarrow$ This is Heuristic Engine.

Core logic: gives a score 0 or 1 based on rules

Rule Based System that uses predefined logic to make decisions. ~~It~~ ^{Project} uses Rule Based Heuristic Engine to

detect patterns, Assigning Scores, and to produce a final score.

Why we use Heuristic? What is the need?

A. Since MuRIL can't capture non-linguistic Cues, we use Heuristic Engine because it is a rule Based System. not just a transformer.

Heuristic captures cues like emojis, slangs, stylistic patterns like non-linguistic humour.

With heuristic we add a real meaning to the model, so, that model captures real world scenarios.

What heuristic do? [core logic]

A. Detects Specific Patterns:

[detects Emojis, slangs, captions, Repetition]

Assign Scores [+ points for ^{Humor} sarcasm, - points for non-humor]

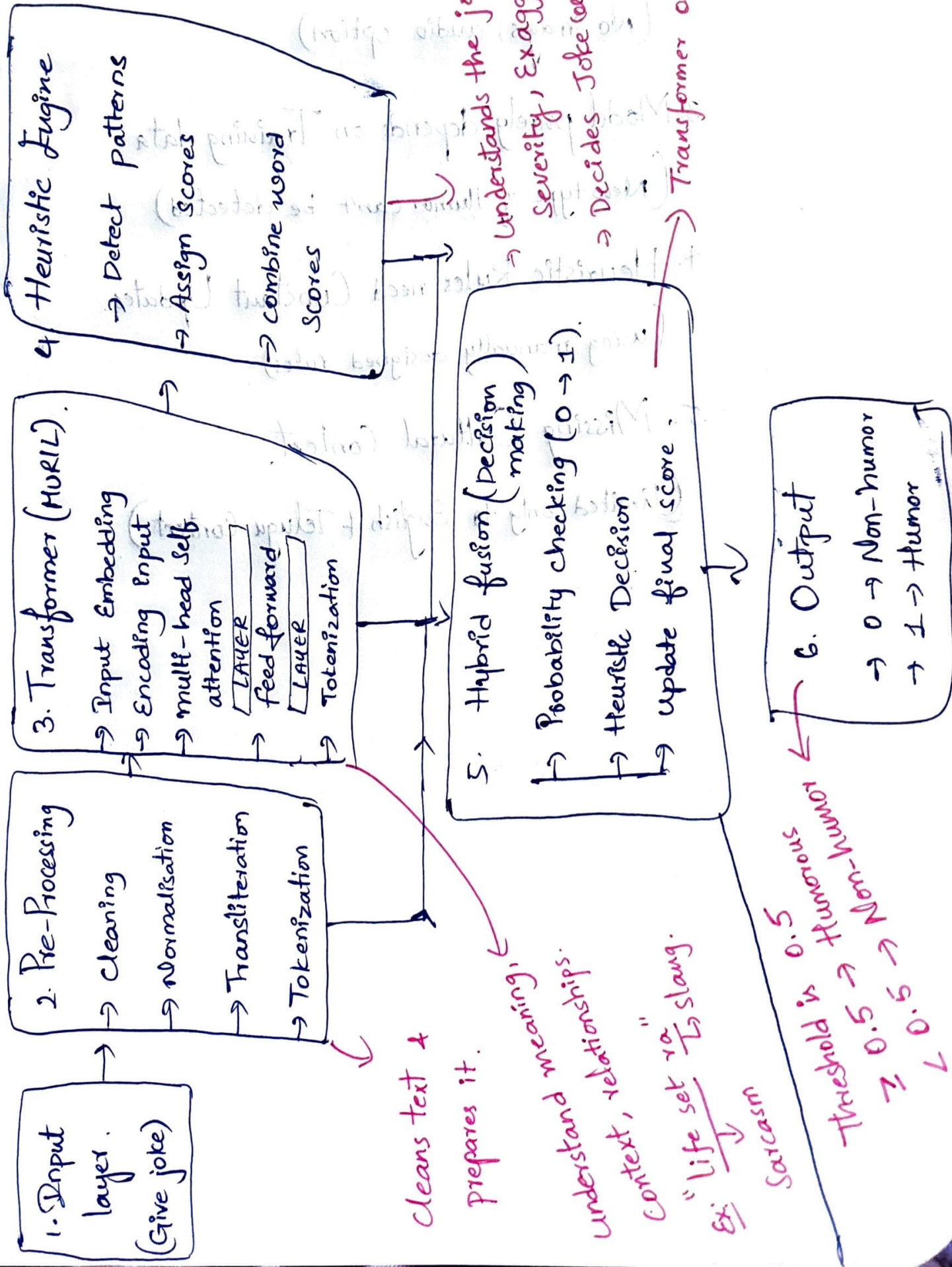
[Evaluate slangs and Emojis for each words]

Combines final Score

≥ 0.30	strong humor
$0.15 - 0.30$	moderate humor
≤ -0.20	non-humor

It mimics human thinking.

Architecture of Project



Limitations: 1. Dataset is artificial (Synthetic dataset)

2. No multi-modal understanding.

(No images, audio option)

3. Model purely depends on Training data.

(New type of humor can't be detected)

4. Heuristic Rules need Constant Updates.

(using manually designed rules)

5. Missing Cultural Context.

(limited only to English & Telugu Context)

Results & Findings :-

Best model : MuRIL + Heuristic $\sim 92.4\%$
(Hybrid model) accuracy.

→ Balanced performance.

We also compared our model to :

MuRIL (Base)	XLM-R	IndicBERTv2	mBERT	MT5
(90.1%)	(88.5%)	(87.9%)	(85.8%)	(81.1%)

→ There are more High True positives (462)

↳ which detect humor well.

And almost Equal High True Negatives (458).

↳ Trains the model well.

→ Making the model Balanced.

On doing Cross Dataset Evaluation, of Accuracy on

$V_3 \rightarrow V_2$ (90.2%) is obtained.

test ↙

↘ train

→ making sure that the model works well on different dataset.

→ And shows & no overfitting is done.

Code Mixed Handling

(CMI) ~ 0.78

→ meaning that the model handles mixed data efficiently.

→ works well for real English data.